

CineSeat Backend — İş Planı

2 Kişi: Ömer & Berke · Mevcut CQRS Mimarisi (5 Proje, Repository Pattern, MediatR)

Amaç: backend'in kalanını iki kişiyle, aynı konvansiyonları koruyarak fazlar halinde tamamlamak. **MediatR ve repository kalıplarına birebir uyulmalıdır** — Bölüm 1 zorunlu referanstır.

0. Mevcut Durum

```
backend/
├─ Core/
│   ├── CineSeat.Domain      → Entities, Enums (BaseEntity: Id, CreatedAt, UpdatedAt, IsDeleted)
│   └── CineSeat.Application  → Features (CQRS), Common, Repositories (arayüzler) [EF YOK]
├─ Infrastructure/
│   ├── CineSeat.Infrastructure → şu an ~boş (dış servisler buraya: JWT, mail, ödeme...)
│   └── CineSeat.Persistence   → DbContext, Migrations, Repository impl, Interceptor, ServiceRegistration
└─ Presentation/
    └── CineSeat.WebAPI        → Controllers, Middleware, Program.cs
```

Bağımlılık yönü: WebAPI → Application + Persistence + Infrastructure; Persistence → Application + Domain; Application → Domain. (Onion korunuyor; Application EF'i tanımıyor)

Hazır altyapı (dokunmadan kullanılacak):

- MediatR pipeline: `LoggingBehavior` (süre loglar) → `ValidationBehavior` (FluentValidation) → Handler.
- Repository: `IReadRepository<T>` / `IWriteRepository<T>` + entity başına `IXReadRepository` / `IXWriteRepository`.
- `IAsyncQueryExecutor` — Application'ın EF'siz sorgu materyalize etmesini sağlar (ToListAsync/CountAsync/AnyAsync/FirstOrDefaultAsync).
- `AuditableEntityInterceptor` — CreatedAt/UpdatedAt'i **otomatik** doldurur.
- Global **soft-delete filter** — IsDeleted==true kayıtlar sorgularda görünmez.
- `Result` / `Result<T>`, `PagedResult<T>` (henüz kullanılmıyor), `NotFoundException` / `ValidationException`, `app.UseExceptionHandler()`.

Tamamlanan: Cities (CreateCity, GetAllCities), Movies (CreateMovie + validator, GetMovies).

Eksikler: ~19 entity feature, **auth/JWT'nin tamamı**, çoğu validator, pagination, RBAC, seed data.

1. Konvansiyonlar — "Bir Feature Nasıl Eklenir" (MediatR odaklı, ZORUNLU)

1a. Entity için repository

`Core/CineSeat.Application/Repositories/<Entity>/`:

```
public interface IDistrictReadRepository : IReadRepository<District> { }
public interface IDistrictWriteRepository : IWriteRepository<District> { }
```

`Infrastructure/CineSeat.Persistence/Repositories/<Entity>/`:

```
public class DistrictReadRepository : ReadRepository<District>, IDistrictReadRepository
{
    public DistrictReadRepository(ApplicationDbContext ctx) : base(ctx) { }
}
public class DistrictWriteRepository : WriteRepository<District>, IDistrictWriteRepository
{
    public DistrictWriteRepository(ApplicationDbContext ctx) : base(ctx) { }
}
```

`Persistence/ServiceRegistration.cs` içine:

```
services.AddScoped<IDistrictReadRepository, DistrictReadRepository>();
services.AddScoped<IDistrictWriteRepository, DistrictWriteRepository>();
```

1b. Command (yazma)

```
public class CreateDistrictCommand : IRequest<long>
{
    public string DistrictName { get; set; } = string.Empty;
    public long CityId { get; set; }
}

public class CreateDistrictCommandHandler : IRequestHandler<CreateDistrictCommand, long>
{
    private readonly IDistrictWriteRepository _write;
    public CreateDistrictCommandHandler(IDistrictWriteRepository write) => _write = write;

    public async Task<long> Handle(CreateDistrictCommand request, CancellationToken ct)
    {
        var district = new District { DistrictName = request.DistrictName, CityId = request.CityId };
        await _write.AddAsync(district, ct);
        await _write.SaveAsync(ct);
        return district.Id; // CreatedAt'i ELLE SET ETME - interceptor doldurur
    }
}

public class CreateDistrictCommandValidator : AbstractValidator<CreateDistrictCommand>
{
    public CreateDistrictCommandValidator()
    {
        RuleFor(x => x.DistrictName).NotEmpty().HasMaxLength(100);
        RuleFor(x => x.CityId).GreaterThan(0);
    }
}
```

1c. Query (okuma) — EF'in `ToListAsync`'ini DEĞİL, `IAsyncQueryExecutor`'ı kullan

```

public class GetDistrictsByCityQuery : IRequest<List<DistrictDto>>
{
    public long CityId { get; set; }
}

public class GetDistrictsByCityQueryHandler
    : IRequestHandler<GetDistrictsByCityQuery, List<DistrictDto>>
{
    private readonly IDistrictReadRepository _read;
    private readonly IAsyncQueryExecutor _exec;
    public GetDistrictsByCityQueryHandler(IDistrictReadRepository read, IAsyncQueryExecutor exec)
    { _read = read; _exec = exec; }

    public async Task<List<DistrictDto>> Handle(GetDistrictsByCityQuery request, CancellationToken ct)
    {
        var query = _read.GetWhere(d => d.CityId == request.CityId, tracking: false)
            .Select(d => new DistrictDto { Id = d.Id, DistrictName = d.DistrictName });
        return await _exec.ToListAsync(query, ct);
    }
}

```

1d. DTO (class) · 1e. Controller (IMediator.Send)

```

[ApiController]
[Route("api/[controller]")]
public class DistrictsController : ControllerBase
{
    private readonly IMediator _mediator;
    public DistrictsController(IMediator mediator) => _mediator = mediator;

    [HttpPost] public async Task<IActionResult> Create([FromBody] CreateDistrictCommand c)
        => Ok(await _mediator.Send(c));
    [HttpGet] public async Task<IActionResult> GetByCity([FromQuery] long cityId)
        => Ok(await _mediator.Send(new GetDistrictsByCityQuery { CityId = cityId }));
}

```

MediatR Altın Kuralları

Kural	Açıklama
Command/Query = class : IRequest<T>	T dönüş tipidir (id, DTO, List<DTO>, Result<T>...)
Handler = IRequestHandler<TReq, TRes>	Ctor'da repository + IAsyncQueryExecutor enjekte edilir
DI'a elle EKLEME	AddApplicationServices assembly'i tarar → yeni handler/validator otomatik bulunur
Pipeline sırası	Logging → Validation → Handler (dokunulmaz)
Controller ince	Sadece _mediator.Send(...), iş mantığı yok
Okuma IAsyncQueryExecutor	Handler'da using Microsoft.EntityFrameworkCore OLMAMALI
CreatedAt/UpdatedAt	Elle set edilmez — interceptor doldurur
Hata	throw new NotFoundException(...) / ValidationException → middleware çevirir

2. İş Bölümü (2 Kişi)

	Ömer — Kimlik + Katalog + Sosyal	Berke — Konum + Mekan + Rezervasyon
Zor alan	Auth & RBAC (güvenlik)	Rezervasyon akışı (eşzamanlılık, transaction)
Entity'ler	User, Role, Permission, RolePermission, Movie(tamamla), Genre, MovieGenre, Campaign, UserFavorite, Comment	City(tamamla), District, Cinema, Hall, Technology, HallTech, Seat, Showtime, SeatLock, Reservation, Ticket

3. Fazlar

Faz 1 — Auth (Ömer) + Konum & Sinema (Berke) paralel

Ömer — Auth (kritik yol):

- Arayüzler Application'da: **ITokenService**, **IPasswordHasher**, **ICurrentUserService**
- Impl'ler **CineSeat.Infrastructure** (boş proje bunun için): JWT (JwtBearer), PBKDF2/BCrypt hash → Infrastructure/ServiceRegistration + Program.cs
- **Features/Auth/Commands/Register** + **Login** (Command+Handler+Validator), dönüş AuthResult (token + kullanıcı)
- Program.cs: AddAuthentication(JwtBearer) + AddAuthorization + UseAuthentication
- Rol/izin **seed** (Admin, User) — Persistence'ta DbInitializer
- ICurrentUserService (WebAPI, IHttpContextAccessor)

Berke — Konum & Sinema:

- City tamamla: UpdateCity, DeleteCity, GetCityById
- District: Create/Update/Delete/GetDistrictsByCity (+ read/write repo + register)
- Cinema: Create/Update/Delete/GetById/GetByCity (+ repo)
- Her feature Bölüm 1 reçetesiyle; en az bir command'a validator (pratik)

Hedef: Login token üretiyor; Berke pattern'e hakim; konum+sinema tam.

Faz 2 — Katalog (Ömer) + Salon/Koltuk/Seans (Berke)

Ömer — Katalog:

- Movie tamamla: Update, Delete, GetMovieById, GetMovies'e **filtre + pagination** (PagedResult<MovieDto>)
- Genre: CRUD · MovieGenre: AssignGenreToMovie / RemoveGenre / GetGenresOfMovie
- Campaign: CRUD + GetActiveCampaigns

Berke — Mekan & Seans:

- Hall: CRUD (Cinema'ya bağlı)
- Technology: CRUD · HallTech: AssignTechToHall / RemoveTech / GetTechsOfHall
- Seat: CreateSeats (toplu üretim), GetSeatMap (salona göre)
- Showtime: Create/Update/Delete/GetById/GetByMovie/GetByCinema (Movie[Ömer] + Hall[kendi] FK)

Hedef: katalog + mekan + seanslar tam sorgulanabilir.

Faz 3 — Rezervasyon (Berke, EN ZOR) + Sosyal & Profil (Ömer)

Berke — Rezervasyon:

- ☐ SeatLock: LockSeat, ReleaseSeat — Showtimeld+SeatId **unique index** çift kilidi engeller; çakışmada ConflictException
- ☐ Reservation: CreateReservation — koltuk doğrulama, fiyat = seans BasePrice × koltuklar – kampanya indirimi, **tek transaction** (AddRangeAsync + tek SaveAsync)
- ☐ GetMyReservations (current user), CancelReservation
- ☐ Ticket: IssueTicket (rezervasyon onayında), GetTicketByld

Ömer — Sosyal & Profil:

- ☐ UserFavorite: Add/Remove/GetMyFavorites (current user)
- ☐ Comment: Add/Delete/GetCommentsByMovie (+ Movie.AvgScore güncelle)
- ☐ User: GetProfile, UpdateProfile
- ☐ RBAC: `[Authorize(Roles="Admin")]` yazma/silme endpoint'lerine

Hedef: uçtan uca rezervasyon + sosyal özellikler.

Faz 4 — Kalite, Güvenlik, Bitiş birlikte

- ☐ Tüm command'lara validator; tüm liste endpoint'lerine **pagination** (PagedResult<T>)
- ☐ [Authorize] rolleri gözden geçir
- ☐ **Soft-delete kararı** uygula (Bölüm 5)
- ☐ Seed genişletme: admin user, türler, şehir/ilçe
- ☐ **Swagger'a JWT** (Authorize butonu) + tüm endpoint'leri test
- ☐ Uçtan uca smoke test + kısa README

Bölüşüm: **Ömer** güvenlik/auth/Swagger-JWT · **Berke** pagination/seed/liste standardı · ikisi cross-review.

4. Bağımlılık & Sıra

- **Berke → Ömer'i bekler:** korumalı endpoint'lere [Authorize] için Faz 1 auth (ama feature'ları auth'suz yazıp sonra kilitleyebilir).
- **Showtime (Berke) → Movie (Ömer):** seans filmi FK'lar; Ömer'in Movie'si veya seed film yeterli.
- **Reservation (Berke) → Showtime + Seat (kendi) + Campaign (Ömer):** kampanya indirimi Ömer'e bağlı — gelene kadar indirimsiz yazılabilir.
- **Kritik yol:** Faz 1 Auth → korumalı özellikler; Showtime → Reservation.

5. Karara Bağlanacak Açık Noktalar

1. **Soft-delete vs hard-delete:** `WriteRepository.Remove/RemoveAsync` şu an **hard delete** yapıyor, ama global filter + `IsDeleted` **soft-delete** ima ediyor. Öneri: *soft-delete* (`IsDeleted=true` + `Update`) — filter zaten hazır.
2. **Dış servis yeri:** JWT/hashing → `CineSeat.Infrastructure`. Arayüz `Application`, impl `Infrastructure`, DI `Infrastructure`.
3. **ICurrentUserService:** arayüz `Application`, impl `WebAPI` (`IHttpContextAccessor`).
4. **Pagination standardı:** liste endpoint'leri `List<Dto>` mı `PagedResult<Dto>` mı? Öneri: *büyük listeler* `PagedResult`.
5. **Result<T> vs exception:** beklenen iş-kuralı hataları `Result.Failure` mı, `exception` mı? Öneri: *beklenen hatalar* `Result`, *gerçek istisnalar* `exception`.

6. Git Akışı & Definition of Done

```
main (korumalı)
├─ feature/omer-auth-catalog-social
└─ feature/berke-location-venue-booking
```

- Herkes kendi `Features/<Entity>/` ve `Controllers/` dosyalarında → çakışma minimum.
- Ortak dosyalar (`ServiceRegistration.cs` repo kaydı, `Program.cs`) küçük/dikkatli düzenlenir; sık main çek.
- PR → en az 1 review → main.

Definition of Done (her feature): Command/Query + Handler ✓ · (write ise) Validator ✓ · read/write repo + register ✓ · DTO ✓ · Controller endpoint ✓ · `IAsyncQueryExecutor` kullanıldı (query'de EF yok) ✓ · Swagger'dan test ✓ · PR review ✓

Kaba Zaman Çizelgesi

Faz	Ömer	Berke	Süre
1	Auth + RBAC iskeleti	Konum + Sinema	~4-5 gün
2	Katalog (Movie/Genre/Campaign)	Mekan + Seans	~4-6 gün
3	Sosyal + Profil	Rezervasyon + Bilet	~5-7 gün
4	Güvenlik/Swagger	Pagination/Seed	~2-3 gün
Toplam			~3-4 hafta